

RƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP
KHOA ĐIỆN TỬ
BỘ MÔN: CÔNG NGHỆ THÔNG TIN

★ ★ ★



BÀI TIỂU LUẬN KIỂM THỬ PHẦN MỀM

ĐỀ TÀI: KIỂM THỬ API WEB TỪ GÓC NHÌN CHIẾN LƯỢC KIỂM THỬ

Giảng viên hướng dẫn	:	Ts. Nguyễn Tiến Linh
Họ tên	:	Dương Thị Ly
MSV	:	K225480106045
Lớp	:	K58KTPM

Thái Nguyên 2026

MỤC LỤC

DANH MỤC HÌNH ẢNH.....	3
LỜI NÓI ĐẦU	4
CHƯƠNG 1: MỞ ĐẦU	5
1.1 Lý do chọn đề tài.....	5
1.2 Mục tiêu nghiên cứu	5
1.3 Phạm vi nghiên cứu	5
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN	7
2.1. Tổng quan về Web API và Chiến lược Kiểm thử API	7
2.1.1. Định nghĩa Web API (RESTful)	7
2.1.2. Định nghĩa API Testing.....	7
2.1.3. Chiến lược kiểm thử API trong Tháp kiểm thử (Testing Pyramid)	8
2.2. Phân tích chuyên sâu 5 trụ cột của Chiến lược API Testing	9
2.2.1. Kiểm thử Hợp đồng (Contract Testing)	9
2.2.2. Kiểm thử hướng dữ liệu (Data-driven Testing).....	10
2.2.3. Xác thực và Phân quyền (Authentication & Authorization)	10
2.2.4. Kiểm thử lỗi biên và Xử lý ngoại lệ (Boundary & Error Handling).....	12
2.2.5. Tích hợp liên tục (Continuous Integration - CI)	13
2.3. Sự liên hệ với chất lượng phần mềm	14
CHƯƠNG 3: VẬN DỤNG VÀ THỰC HÀNH TẠI NGŨ CẢNH CỤ THỂ.....	15
3.1. Mô tả bài toán ngữ cảnh thực tế.....	15
3.1.1. Tổng quan về hệ thống giả lập	15
3.1.2. Danh sách các API endpoints được chọn để Demo	15
3.3. Kết quả và Đánh giá.....	25
CHƯƠNG 4: KẾT LUẬN	27
4.1. Tổng kết nội dung nghiên cứu	27
4.2. Hạn chế của giải pháp	28
4.3. Định hướng mở rộng đề tài.....	28
TÀI LIỆU THAM KHẢO	30

DANH MỤC HÌNH ẢNH

Hình 3.1. Giao diện Tests contract trong Postman	18
Hình 3.2 Test Data trên postman	19
Hình 3.3 Test Authentication	21
Hình 3.4. Test Authorization.....	21
Hình 3.5 Response từ Postman cho hai trường hợp lỗi biên 0 và 101	23
Hình 3.6 Giao diện kiểm thử biên trên postman	25

LỜI NÓI ĐẦU

Trong kỷ nguyên số hóa và sự bùng nổ của kiến trúc hệ thống hiện đại, ứng dụng web không còn là những khối mã nguồn đóng kín (Monolithic Architecture). Thay vào đó, xu hướng chuyển dịch sang kiến trúc Microservices và việc bên thứ ba tích hợp dịch vụ đã biến API (Application Programming Interface) trở thành mạch máu kết nối, quyết định sự vận hành mượt mà của toàn bộ hệ thống phần mềm. API ổn định, hiệu năng và bảo mật đồng nghĩa với việc trải nghiệm người dùng được đảm bảo và giá trị kinh doanh của doanh nghiệp được duy trì.

Tuy nhiên, việc kiểm thử API thường đối mặt với nhiều thách thức lớn do tính chất không có giao diện trực quan (UI-less) và sự phức tạp trong việc quản lý trạng thái dữ liệu, cơ chế xác thực hay sự thay đổi liên tục của các phiên bản. Nếu chỉ kiểm thử API một cách rời rạc, cảm tính mà thiếu đi một tầm nhìn tổng thể, hệ thống rất dễ gặp phải các lỗi hổng nghiêm trọng khi vận hành thực tế. Chính vì vậy, việc tiếp cận API testing dưới một góc nhìn chiến lược là điều kiện tiên quyết để tối ưu hóa chi phí, thời gian và nâng cao chất lượng sản phẩm.

Nhận thức được tầm quan trọng đó, em đã quyết định thực hiện đề tài nghiên cứu cho môn học Kiểm thử phần mềm với đề tài **"Kiểm thử API Web từ góc nhìn chiến lược kiểm thử"**.

Mặc dù đã có nhiều cố gắng trong quá trình tìm hiểu và xây dựng nội dung, song do giới hạn về mặt thời gian và kinh nghiệm thực tiễn, bài báo cáo chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp, nhận xét từ Thầy Cô để đề tài được hoàn thiện và có giá trị ứng dụng cao hơn.

Em xin chân thành cảm ơn!

CHƯƠNG 1: MỞ ĐẦU

1.1 Lý do chọn đề tài

- Sự dịch chuyển mạnh mẽ từ kiến trúc Monolithic sang Microservices và Single Page Applications (SPA) khiến Web API trở thành xương sống của các hệ thống phần mềm hiện đại.
- Kiểm thử ở tầng UI (End-to-End) bộc lộ nhiều hạn chế (chậm, dễ gây - flaky, chi phí cao). Kiểm thử API nằm ở tầng giữa của tháp kiểm thử (Testing Pyramid), mang lại hiệu năng cao và khả năng phát hiện lỗi sớm hơn (Shift-Left Testing).
- Việc kiểm thử API mong muốn, thiếu chiến lược sẽ dẫn đến bỏ sót các lỗi nghiêm trọng về logic dữ liệu, bảo mật (xác thực) và tích hợp. Vì vậy, việc thiết kế một chiến lược toàn diện (Contract, Data, Auth, Boundary, CI) là vô cùng cấp thiết.

1.2 Mục tiêu nghiên cứu

- Xây dựng chiến lược Kiểm thử giao ước (Contract Testing)
- Thiết kế chiến lược Quản lý dữ liệu kiểm thử (Data-driven Testing)
- Chuẩn hóa quy trình Kiểm thử lỗi biên và kiểm soát ngoại lệ (Boundary & Exception Testing)
- Xây dựng khung Kiểm thử xác thực và phân quyền (Authentication & Authorization Testing)
- Tự động hóa và Tích hợp kiểm thử vào chu trình CI/CD (Continuous Integration)

1.3 Phạm vi nghiên cứu

1.3.1 Đối tượng nghiên cứu

- Đối tượng công nghệ: Tập trung nghiên cứu kiến trúc Web API phổ biến hiện nay, trọng tâm là RESTful API (sử dụng các phương thức HTTP chuẩn như GET, POST, PUT, DELETE và định dạng dữ liệu JSON).
- Đối tượng quy trình: Quy trình thiết kế chiến lược, xây dựng kịch bản kiểm thử tự động (Automation Testing) ở tầng dịch vụ (Service/Integration Layer) và tích hợp vào quy trình phát triển phần mềm liên tục.

1.3.2 Nội dung nghiên cứu (In-scope)

Đề tài tập trung nghiên cứu sâu vào việc xây dựng chiến lược và triển khai kỹ thuật cho 5 thành phần cốt lõi:

- Chiến lược Kiểm thử Giao ước (Contract Testing): Nghiên cứu việc kiểm thử cấu trúc (Schema Validation), kiểu dữ liệu và định dạng của Request/Response để đảm bảo tính tương thích giữa Client và Server.
- Chiến lược Dữ liệu kiểm thử (Data Management): Nghiên cứu phương pháp kiểm thử hướng dữ liệu (Data-driven Testing), cách tổ chức, tạo lập và làm sạch dữ liệu test tự động (bao gồm cả dữ liệu hợp lệ và không hợp lệ).
- Kiểm thử lỗi biên (Boundary & Exception Testing): Nghiên cứu hành vi của API tại các điểm giới hạn của tham số đầu vào (ví dụ: độ dài chuỗi, khoảng giá trị số, định dạng dữ liệu đặc biệt) và cách hệ thống xử lý mã lỗi (Error Handling).
- Kiểm thử Xác thực và Phân quyền (Auth & Security): Nghiên cứu kịch bản kiểm thử cho các cơ chế bảo mật API phổ biến (Token-based như JWT, OAuth2, API Keys) và kiểm thử phân quyền kiểm soát truy cập (như phân quyền theo vai trò - RBAC).
- Tích hợp hệ thống CI/CD: Nghiên cứu mô hình tự động hóa luồng chạy test (Test Automation Execution) kết hợp với các công cụ CI/CD để kích hoạt kiểm thử tự động mỗi khi có thay đổi mã nguồn (Commit/Pull Request).
- Công cụ minh họa: Postman, Newman, GitHub Actions (hoặc công cụ tương đương được chỉ định trong tài liệu tham khảo của bạn).

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN

2.1. Tổng quan về Web API và Chiến lược Kiểm thử API

2.1.1. Định nghĩa Web API (RESTful)

- API (Application Programming Interface - Giao diện lập trình ứng dụng) là một tập hợp các quy tắc, giao thức và định nghĩa cho phép các thành phần phần mềm khác nhau tương tác, trao đổi dữ liệu với nhau. Khi được triển khai trên môi trường mạng thông qua kiến trúc REST (Representational State Transfer) và giao thức HTTP, hệ thống được gọi là RESTful API.
- RESTful API tổ chức hệ thống dưới dạng các tài nguyên (Resources) được định danh bằng các đường dẫn URL độc lập (endpoints). Việc tương tác với tài nguyên được thực hiện thông qua bốn phương thức HTTP chuẩn hóa (HTTP Methods):
 - GET: Đọc hoặc lấy thông tin tài nguyên từ hệ thống (không làm thay đổi trạng thái dữ liệu).
 - POST: Tạo mới một tài nguyên hệ thống (ví dụ: gửi dữ liệu lên endpoint /api/products để khởi tạo sản phẩm).
 - PUT/PATCH: Cập nhật tài nguyên đã tồn tại trên hệ thống.
 - DELETE: Loại bỏ hoàn toàn một tài nguyên khỏi hệ thống.
- Dữ liệu trao đổi giữa Client và Server trong RESTful API thường sử dụng định dạng JSON (JavaScript Object Notation) nhờ tính gọn nhẹ, dễ đọc và tương thích tốt với hầu hết mọi ngôn ngữ lập trình hiện đại.

2.1.2. Định nghĩa API Testing

- API Testing (Kiểm thử API) là một nhánh của kiểm thử phần mềm tập trung vào việc xác thực trực tiếp tầng logic nghiệp vụ (Business logic layer/Service layer) của ứng dụng mà không cần thông qua giao diện đồ họa người dùng (UI).
- Mục tiêu cốt lõi của API Testing bao gồm việc đảm bảo API hoạt động đúng theo tài liệu đặc tả kỹ thuật, có khả năng xử lý dữ liệu chính xác, đảm bảo các cơ chế

an toàn bảo mật, đồng thời duy trì tính ổn định và khả năng phục hồi lỗi của hệ thống khi đối mặt với các điều kiện dữ liệu bất thường.

2.1.3. Chiến lược kiểm thử API trong Tháp kiểm thử (Testing Pyramid)

Trong mô hình Tháp kiểm thử kinh điển của Mike Cohn, kiểm thử được chia thành ba tầng rõ rệt: Unit Test (Kiểm thử đơn vị) ở đáy, API/Integration Test (Kiểm thử tích hợp) ở giữa, và UI/End-to-End Test (Kiểm thử giao diện) ở đỉnh.

- Sự tối ưu của Kiểm thử API so với UI và sự hỗ trợ cho Unit Test
 - Tối ưu hơn UI Test về chi phí và thời gian: Kiểm thử ở tầng giao diện (UI) thường thực thi rất chậm do phải chờ đợi trình duyệt render hình ảnh và các phần tử DOM. UI Test cũng rất dễ bị gây (Flaky tests) khi có sự thay đổi nhỏ về thiết kế giao diện (CSS, ID phần tử). Ngược lại, API Test giao tiếp trực tiếp qua giao thức HTTP bằng dữ liệu thô nên tốc độ thực thi nhanh hơn gấp hàng chục lần, cho phép phát hiện lỗi logic hệ thống ngay từ giai đoạn sớm (Shift-Left Testing) khi UI thậm chí còn chưa được thiết kế.
 - Hỗ trợ hoàn hảo cho Unit Test: Trong khi Unit Test chỉ cô lập và kiểm tra tính đúng đắn của từng hàm/lớp riêng lẻ từ phía lập trình viên, nó hoàn toàn bất lực trước các lỗi phát sinh do sự tương tác giữa các thành phần khác nhau. API Testing đóng vai trò như một cầu nối tích hợp, xác thực xem các module mã nguồn khi ghép lại với nhau, kết nối với cơ sở dữ liệu và các dịch vụ bên ngoài có vận hành trơn tru hay không.
- Bộ khung phương pháp kiểm thử có hệ thống:
 - Một chiến lược kiểm thử API toàn diện không chỉ dừng lại ở việc kiểm tra phản hồi đúng/sai (Happy path) mà phải là một tập hợp các phương pháp có tính hệ thống cao, bao gồm 5 cấu phần xương sống: Kiểm thử giao ước (Contract), Kiểm thử hướng dữ liệu (Data-driven), Xác thực và phân quyền (Auth), Kiểm thử lỗi biên (Boundary), và Tự động hóa qua CI/CD. Các trụ cột này phối hợp chặt chẽ tạo thành một màng lọc chất lượng đa tầng cho hệ thống Web API.

2.2. Phân tích chuyên sâu 5 trụ cột của Chiến lược API Testing

2.2.1. Kiểm thử Hợp đồng (Contract Testing)

- Khái niệm: Là phương pháp kiểm thử nhằm xác thực xem API có tuân thủ đúng cam kết về mặt cấu trúc dữ liệu đã được thống nhất giữa bên cung cấp dịch vụ (Provider/Backend) và bên tiêu thụ dịch vụ (Consumer/Frontend) hay không. Nội dung kiểm tra bao gồm: endpoint chính xác, phương thức HTTP phù hợp, sự tồn tại của các trường dữ liệu và kiểu dữ liệu của chúng.
- Công cụ phổ biến: Pact (chuyên dụng cho Microservices), Postman Schema Validation (sử dụng thư viện Ajv tích hợp sẵn).
- Ưu điểm: Giúp phát hiện cực kỳ sớm các sai lệch cấu trúc dữ liệu ngay trong quá trình phát triển độc lập, ngăn chặn hoàn toàn các lỗi sập hệ thống dây chuyền khi tích hợp (Breaking changes).
- Cơ chế hoạt động của JSON Schema Validation:

JSON Schema Validation hoạt động dựa trên nguyên lý so khớp cấu trúc (Structural Matching). Thay vì kiểm tra giá trị cụ thể của dữ liệu (ví dụ: kiểm tra tên sản phẩm có phải là "Iphone 15" hay không), kỹ sư kiểm thử sẽ định nghĩa một bộ khung quy chuẩn (Schema) mô tả định dạng của dữ liệu.

Khi API trả về một Response dạng JSON, công cụ kiểm thử (như Postman) sẽ sử dụng một bộ phân tích cú pháp (Validator) để đối chiếu tự động. Hệ thống sẽ kiểm tra:

- Các trường bắt buộc (required) có xuất hiện đầy đủ hay không.
- Kiểu dữ liệu của từng trường (type: string, number, boolean, object, array) có trùng khớp không.

Nếu Backend tự ý thay đổi tên trường (ví dụ: từ quantity thành qty) hoặc đổi kiểu dữ liệu (từ số sang chuỗi), trình kiểm duyệt Schema sẽ ngay lập tức phát hiện và đánh dấu kịch bản kiểm thử thất bại (Failed), giúp Frontend không bị bất ngờ khi gọi API thực tế.

2.2.2. Kiểm thử hướng dữ liệu (Data-driven Testing)

- Nguyên lý tách biệt giữa kịch bản kiểm thử (Test Scripts) và tập dữ liệu (Test Data)
 - Trong kiểm thử truyền thống, dữ liệu thường được viết cứng (Hard-coded) vào trong mã kiểm thử, dẫn đến việc phải sao chép lặp lại một đoạn code nhiều lần nếu muốn kiểm tra nhiều trường hợp dữ liệu khác nhau.
 - Nguyên lý cốt lõi của Data-driven Testing (DDT) là tách biệt hoàn toàn phần logic kiểm thử (các câu lệnh assertion gửi request, kiểm tra mã trạng thái) ra khỏi bộ dữ liệu đầu vào. Bộ dữ liệu này được lưu trữ riêng biệt dưới dạng các tệp tin cấu trúc như CSV hoặc JSON. Khi thực thi, công cụ kiểm thử sẽ đọc tệp dữ liệu và lặp lại (Iterate) duy nhất một kịch bản kiểm thử với các tham số thay đổi theo từng dòng của tệp dữ liệu.
- Cách quản lý tổ hợp dữ liệu (Ma trận dữ liệu đúng/sai) để tối ưu độ bao phủ:
 - Để tối ưu hóa độ bao phủ của các ca kiểm thử mà không làm phình to số lượng kịch bản, kỹ sư kiểm thử xây dựng một Ma trận dữ liệu (Data Matrix) chứa cả hai trường hợp:
 - Dữ liệu hợp lệ (Positive Data): Chứa các giá trị chuẩn, nằm trong vùng xử lý của hệ thống, kỳ vọng nhận về các mã thành công như 200 OK hoặc 201 Created.
 - Dữ liệu không hợp lệ (Negative Data): Chứa các giá trị sai định dạng, thiếu trường, dữ liệu rỗng, kỳ vọng hệ thống chặn lại và trả về các mã lỗi chính xác như 400 Bad Request hoặc 422 Unprocessable Entity.
- Việc chạy luồng lặp qua ma trận dữ liệu này giúp kiểm tra khả năng chịu lỗi bền bỉ của một Endpoint đơn lẻ trước hàng loạt tổ hợp dữ liệu đầu vào khác nhau.

2.2.3. Xác thực và Phân quyền (Authentication & Authorization)

Vì API phơi bày trực tiếp các cổng kết nối ra môi trường mạng, việc thiết lập rào chắn bảo mật là bắt buộc:

- Cơ chế bảo mật API phổ biến:

- API Keys: Một chuỗi ký tự bí mật duy nhất được truyền qua Header hoặc Query Parameter để định danh ứng dụng đang gọi API.
- JWT Token (JSON Web Token): Cơ chế xác thực phi trạng thái (Stateless). Sau khi người dùng đăng nhập thành công, Server cấp một chuỗi mã hóa chứa thông tin định danh (Payload) và chữ ký số. Client sẽ đính kèm chuỗi JWT này vào Header Authorization: Bearer <token> trong các yêu cầu tiếp theo để chứng minh danh tính.
- OAuth2: Giao thức ủy quyền tiêu chuẩn công nghiệp, cho phép ứng dụng bên thứ ba truy cập tài nguyên thay mặt người dùng thông qua các Access Token có thời hạn mà không cần biết mật khẩu của người dùng.
- Rủi ro bảo mật tầng API và chiến lược kiểm thử phân quyền chéo (Role-based Testing)
 - Lỗ hổng nguy hiểm bậc nhất trên tầng API theo bảng xếp hạng OWASP là BOLA (Broken Object Level Authorization) hay còn gọi là IDOR (Insecure Direct Object Reference). Lỗ hổng này xảy ra khi kẻ tấn công đã xác thực thành công vào hệ thống với một tài khoản thông thường, nhưng lợi dụng việc thay đổi tham số định danh trên URL (ví dụ: thay đổi id=100 thành id=101) để truy cập trái phép vào dữ liệu của người dùng khác mà Backend không kiểm tra quyền sở hữu đối tượng.
 - Chiến lược kiểm thử phân quyền chéo giữa các Role: Để phát hiện lỗ hổng này, chiến lược kiểm thử yêu cầu thiết lập kịch bản chạy song song với nhiều tài khoản thuộc các nhóm quyền khác nhau (Ví dụ: Admin, Manager, User). Kịch bản kiểm thử sẽ lấy Token của tài khoản User để cố tình gửi yêu cầu đến các API đặc quyền của Admin (như tạo sản phẩm, xóa người dùng). Một chiến lược an toàn yêu cầu API phải chặn đứng hành vi này và trả về chính xác mã trạng thái 403 Forbidden.

2.2.4. Kiểm thử lỗi biên và Xử lý ngoại lệ (Boundary & Error Handling)

- Các tham số đầu vào của API ở mọi vị trí (Query Params trên URL, Path Params định danh, hoặc thuộc tính trong Request Body) đều cần được kiểm thử áp dụng hai kỹ thuật kinh điển:
 - Phân vùng tương đương (Equivalence Partitioning): Chia dữ liệu thành các phân vùng hợp lệ và không hợp lệ. Ví dụ, thuộc tính quantity (số lượng sản phẩm) hợp lệ từ 1 đến 1000. Các phân vùng không hợp lệ sẽ là các số nhỏ hơn 1 hoặc lớn hơn 1000.
 - Phân tích giá trị biên (Boundary Value Analysis): Tập trung kiểm thử tại các điểm nhảy cảm xung quanh biên của phân vùng. Với ví dụ trên, các giá trị biên cần kiểm tra nghiêm ngặt sẽ bao gồm các điểm cực hạn: 0, 1, 2 (biên dưới) và 999, 1000, 1001 (biên trên), kèm theo các giá trị ngoại lệ như số âm hoặc chuỗi ký tự không phải số.
- Tiêu chuẩn thiết kế thông điệp lỗi (Graceful Error Handling): Hệ thống xử lý ngoại lệ chuẩn mực (Graceful Error Handling) yêu cầu API khi gặp lỗi không bao giờ được phép sụp đổ hoàn toàn hoặc trả về lỗi thô từ hệ thống (như văng toàn bộ Stack Trace mã nguồn kèm mã lỗi 500 Internal Server Error – điều này vừa làm giảm trải nghiệm phần mềm, vừa làm lộ thông tin an ninh bảo mật). API phải được cấu hình để bắt giữ các ngoại lệ và trả về thông điệp lỗi có cấu trúc chuẩn hóa:
 - Mã 400 Bad Request: Khi Request sai cú pháp JSON hoặc sai định dạng cơ bản.
 - Mã 422 Unprocessable Entity: Khi dữ liệu đúng cú pháp nhưng vi phạm các quy tắc logic nghiệp vụ hoặc lỗi biên (ví dụ: số lượng sản phẩm nhập vào vượt quá kho hoặc là số âm).
 - Cấu trúc phản hồi lỗi: Phải trả về một đối tượng JSON tường minh chứa thông điệp rõ ràng để phía Frontend có thể đọc hiểu và hiển thị thông báo thân thiện tới người dùng cuối (ví dụ: { "error": "Dữ liệu không hợp lệ", "details": "Số lượng phải lớn hơn 0" }).

2.2.5. Tích hợp liên tục (Continuous Integration - CI)

- Vai trò của tự động hóa kiểm thử API trong quy trình DevOps
 - Trong chu trình phát triển phần mềm theo triết lý DevOps và Agile, mã nguồn của ứng dụng được thay đổi và tích hợp liên tục nhiều lần trong ngày. Việc kiểm thử thủ công (Manual Testing) bằng cách nhấn nút gửi request trên giao diện công cụ hoàn toàn thất bại trước tốc độ này.
 - Tự động hóa kiểm thử API đóng vai trò là một Cổng chất lượng tự động (Quality Gate). Nó cho phép kiểm tra hồi quy (Regression Testing) toàn bộ hệ thống ngay lập tức sau mỗi lần lập trình viên thay đổi code, đảm bảo tính năng mới được thêm vào không làm hỏng (break) các tính năng cũ đang chạy ổn định.
- Cơ chế trigger kiểm thử tự động bằng dòng lệnh (CLI): Để tích hợp vào luồng CI/CD, các bộ kịch bản kiểm thử API (như Postman Collection) phải có khả năng thực thi độc lập không phụ thuộc vào giao diện người dùng. Điều này được thực hiện thông qua công cụ dòng lệnh (Command Line Interface - CLI) như Newman (bộ chạy script dòng lệnh của Postman).
- Cơ chế vận hành của luồng CI tự động (ví dụ qua GitHub Actions) bao gồm:
 - Sự kiện kích hoạt (Event Trigger): Lập trình viên đẩy mã nguồn mới lên kho lưu trữ (Commit/Push) hoặc tạo một yêu cầu hợp nhất mã nguồn (Pull Request).
 - Khởi tạo máy ảo (Runner Provisioning): Hệ thống CI tự động khởi tạo một môi trường máy ảo sạch (Linux/Ubuntu), cài đặt môi trường chạy mã nguồn (Node.js).
 - Kích hoạt thực thi (Execution via CLI): Máy ảo tự động chạy lệnh CLI (ví dụ: `newman run collection.json -e environment.json`) để thực hiện toàn bộ các bài test API đã thiết lập. Nếu tất cả các khẳng định (Assertions) đều vượt qua (Passed), mã nguồn mới sẽ được phép đóng gói và sẵn sàng deploy lên môi trường Production. Ngược lại, nếu có bất kỳ lỗi nào xuất hiện, luồng CI sẽ lập tức bị bẻ gãy (Failed) và gửi cảnh báo khẩn cấp cho đội ngũ phát triển.

2.3. Sự liên hệ với chất lượng phần mềm

Việc thiết kế và triển khai một chiến lược kiểm thử API Web bài bản có tác động trực tiếp và mang tính quyết định đến các chỉ số đo lường chất lượng của một dự án phần mềm:

- Giảm thiểu tối đa chi phí sửa lỗi (Bug Cost): Theo quy luật kinh điển của kỹ nghệ phần mềm, chi phí để sửa một lỗi ở môi trường Production có thể cao gấp hàng trăm lần so với việc tìm thấy nó ở giai đoạn lập trình. API Testing đánh giá trực tiếp logic nghiệp vụ ở giai đoạn rất sớm (Shift-Left). Nhờ đó, các lỗi nghiêm trọng về cấu trúc dữ liệu, rò rỉ phân quyền được chặn đứng ngay tại bàn làm việc của lập trình viên, giúp doanh nghiệp tiết kiệm tài nguyên và bảo vệ uy tín sản phẩm.
- Tăng tốc độ phát hành sản phẩm (Time-to-market): Nhờ cơ chế tự động hóa hoàn toàn thông qua dòng lệnh trong quy trình CI/CD, chu kỳ kiểm thử hồi quy được rút ngắn từ vài ngày xuống còn vài phút. Đội ngũ phát triển hoàn toàn tự tin phát hành các tính năng mới liên tục trong ngày mà không sợ làm ảnh hưởng đến tính toàn vẹn của hệ thống cũ.
- Đảm bảo tính ổn định bền bỉ của hệ thống phân tán: Trong các kiến trúc ứng dụng hiện đại, các dịch vụ phụ thuộc chéo vào nhau rất phức tạp. Bằng việc bám sát 5 trụ cột của chiến lược API Testing, hệ thống được trang bị một bộ lọc vững chắc bảo vệ từ tính tương thích cấu trúc (Contract), tính đa dạng dữ liệu (Data-driven), tính an toàn bảo mật (Auth), cho đến khả năng tự chữa lành và chống chịu các dữ liệu độc hại (Boundary Value). Điều này đảm bảo hệ thống luôn vận hành ổn định, nhất quán và tin cậy cao dưới mọi điều kiện tải thực tế.

CHƯƠNG 3: VẬN DỤNG VÀ THỰC HÀNH TẠI NGŨ CẢNH CỤ THỂ

3.1. Mô tả bài toán ngữ cảnh thực tế

3.1.1. Tổng quan về hệ thống giả lập

- Để minh chứng cho việc áp dụng các chiến lược kiểm thử tự động nâng cao vào quy trình phát triển phần mềm hiện đại, nghiên cứu này lựa chọn ngữ cảnh thực tế là Hệ thống Quản lý Cửa hàng điện tử (E-commerce API). Hệ thống được xây dựng trên kiến trúc RESTful API sử dụng nền tảng Node.js và nền khung (framework) Express.
- Trong các hệ thống thương mại điện tử, các dịch vụ phân tách lỗi đóng vai trò quyết định đến tính toàn vẹn của dữ liệu và trải nghiệm người dùng. Việc kiểm thử thủ công (Manual Testing) đối với hệ thống này thường bộc lộ nhiều hạn chế do tần suất cập nhật mã nguồn cao, dẫn đến rủi ro phát sinh lỗi hồi quy (Regression Bugs) lớn trên các luồng nghiệp vụ phức tạp.

3.1.2. Danh sách các API endpoints được chọn để Demo

Chiến lược kiểm thử trong tiểu luận này tập trung vào hai điểm cuối (endpoints) cốt lõi thuộc phân hệ Xác thực và Quản lý Kho hàng:

- POST /api/v1/auth/login (Xác thực người dùng)
 - Mô tả: Tiếp nhận thông tin định danh của người dùng (Email và Mật khẩu). Nếu thông tin chính xác, hệ thống phản hồi một chuỗi mã xác thực Token định dạng mã hóa bảo mật (JSON Web Token - JWT).
 - Vai trò: Là chốt chặn bảo mật đầu tiên, cung cấp quyền truy cập hợp lệ (Credentials) cho các tiến trình kiểm thử phía sau.
- POST /api/v1/products (Tạo sản phẩm mới - Yêu cầu quyền Admin, có ràng buộc dữ liệu biên).
 - Mô tả: Cho phép khởi tạo một sản phẩm mới trong cơ sở dữ liệu với các thuộc tính bao gồm tên sản phẩm (prodName) và số lượng hàng tồn kho (prodQty).

- Ràng buộc nghiệp vụ: API này đòi hỏi nghiêm ngặt về phân quyền người dùng (Yêu cầu quyền hạn mức Admin thông qua cơ chế Bearer Token). Đồng thời, hệ thống áp dụng các quy tắc kiểm tra nghiêm ngặt về dữ liệu đầu vào (Data Validation) tại các vùng biên của thuộc tính số lượng.

3.2. Áp dụng giải pháp kiểm thử chiến lược vào Ngữ cảnh

Bước 1: Thiết kế và kiểm thử cấu trúc giao ước dữ liệu (Contract Testing)

Contract Testing đảm bảo rằng cấu trúc phản hồi (Response Body) từ API luôn tuân thủ đúng định dạng mà các bên phát triển (Frontend/Mobile/Hệ thống tích hợp thứ ba) đã thỏa thuận trước, ngăn chặn việc thay đổi kiểu dữ liệu đột ngột gây sập hệ thống.

- Định nghĩa khuôn mẫu dữ liệu (JSON Schema) cho phản hồi của API GET /api/v1/products:

Dưới đây là tệp mẫu JSON Schema quy định cấu trúc danh sách sản phẩm trả về, đảm bảo các thuộc tính bắt buộc như id phải là kiểu số, prodName phải là chuỗi văn bản và prodQty phải là số nguyên:

```
{ "$schema": "http://json-schema.org/draft-07/schema#",  
  "title": "ProductListSchema",  
  "type": "array",  
  "items": {  
    "type": "object",  
    "properties": {  
      "id": { "type": "integer" },  
      "prodName": { "type": "string" },  
      "prodQty": { "type": "integer", "minimum": 0 }  
    },  
    "required": ["id", "prodName", "prodQty"]  
  }  
}
```

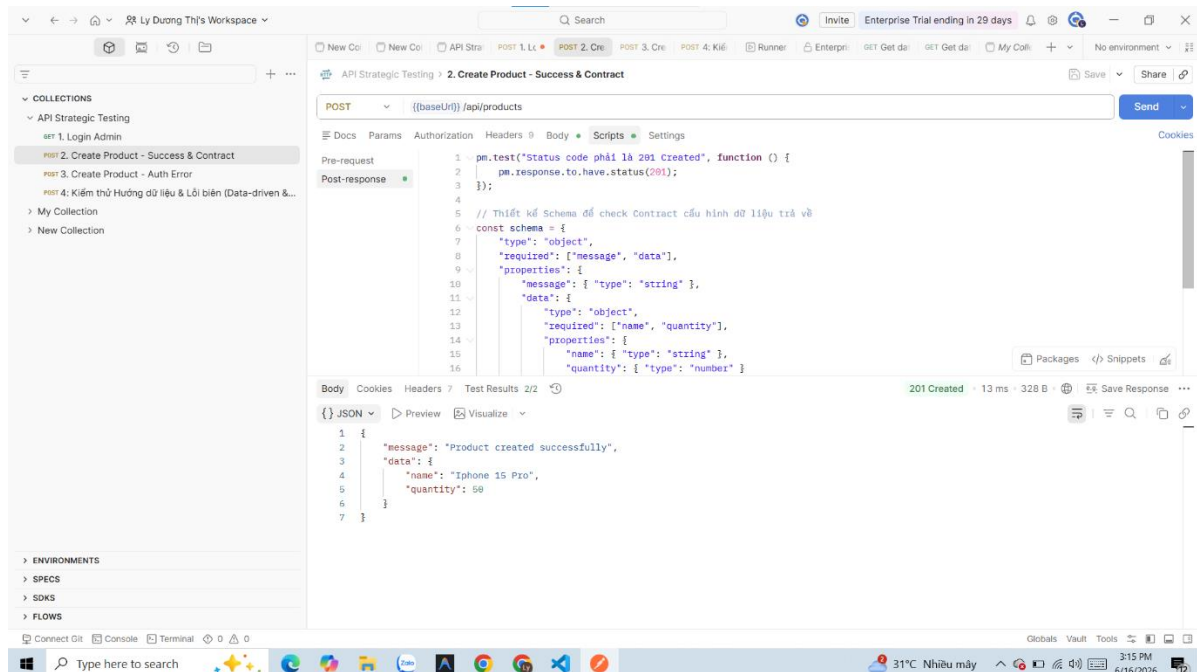

- Kịch bản kiểm thử tự động kiểm tra Giao ước dữ liệu (Postman Script):

Đoạn mã JavaScript dưới đây được nhúng trực tiếp vào Tab Tests của Postman nhằm tự động hóa việc đối chiếu dữ liệu phản hồi thực tế từ máy chủ với khuôn mẫu JSON Schema định sẵn:

```
// Khai báo cấu trúc Giao ước dữ liệu kỳ vọng (JSON Schema)
const productSchema = {
  "type": "array",
  "items": {
    "type": "object",
    "properties": {
      "id": { "type": "integer" },
      "prodName": { "type": "string" },
      "prodQty": { "type": "integer" }
    },
    "required": ["id", "prodName", "prodQty"]
  }
};

// Kiểm thử mức 1: Đảm bảo mã trạng thái phản hồi là 200 OK
pm.test("Status code is 200 OK", function () {
  pm.response.to.have.status(200);
});

// Kiểm thử mức 2: Xác thực cấu trúc dữ liệu trả về khớp hoàn toàn với
// Schema
pm.test("Schema Validation - Kiểm tra giao ước dữ liệu phản hồi khớp
100%", function () {
  var jsonData = pm.response.json();
  pm.expect(tv4.validate(jsonData, productSchema)).to.be.true;
});
```



Hình 3.1. Giao diện Tests contract trong Postman

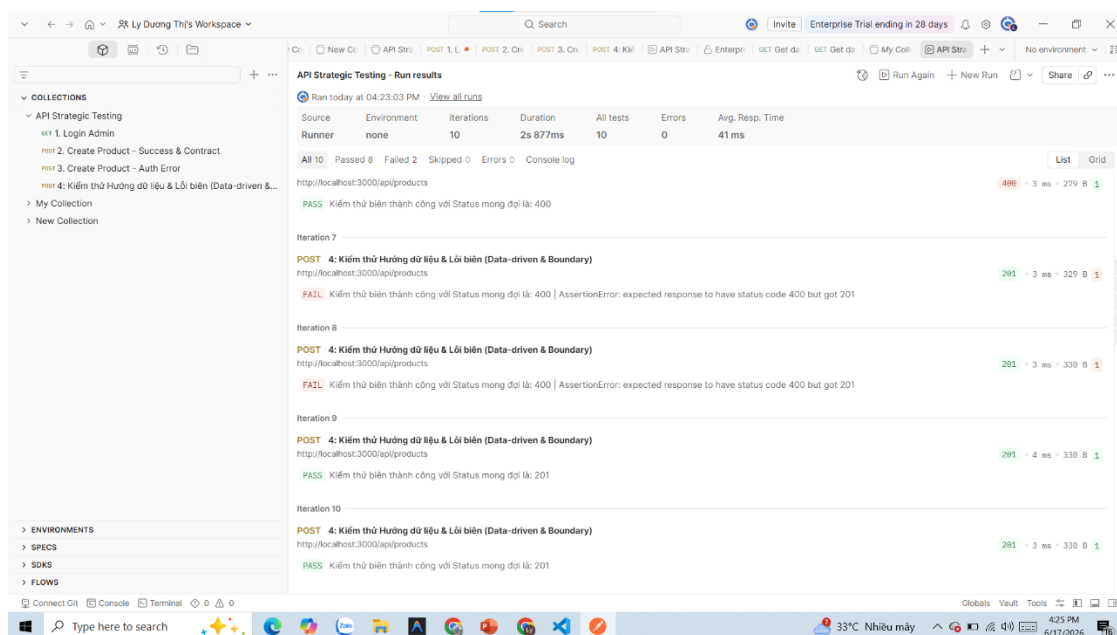
Bước 2: Thiết kế kịch bản hướng dữ liệu (Data-driven Testing)

Để tối ưu hóa độ bao phủ kiểm thử mà không cần nhân bản các yêu cầu (Requests) thủ công, giải pháp Kiểm thử hướng dữ liệu (Data-driven Testing) được áp dụng. Toàn bộ dữ liệu kiểm thử được tách biệt khỏi mã nguồn test script và lưu trữ trong tệp dữ liệu phẳng mở rộng.

Dưới đây là bảng ma trận dữ liệu mẫu cấu hình trong tệp data.csv (gồm 10 trường hợp kiểm thử đặc trưng) nhằm quét toàn diện các kịch bản của tính năng tạo sản phẩm, bao gồm chuỗi rỗng, ký tự đặc biệt, và các giá trị số học phức tạp:

STT	prodName (Tên sản phẩm)	prodQty (Số lượng)	Loại kịch bản (Scenario Type)	Mã lỗi kỳ vọng (Expected Status)
1	iPhone 15 Pro Max	50	Dữ liệu chuẩn hợp lệ	201 Created
2	Samsung Galaxy S24	1	Giá trị biên nhỏ nhất hợp lệ	201 Created

3	MacBook Air M3	100	Giá trị biên lớn nhất hợp lệ	201 Created
4		10	Lỗi: Tên sản phẩm để trống	400 Bad Request
5	Bàn phím Cơ @# \$! 2026	5	Tên chứa ký tự đặc biệt hợp lệ	201 Created
6	Chuột không dây Logitech	0	Lỗi: Dưới biên nhỏ nhất (0)	400 Bad Request
7	Tai nghe Sony WH-1000XM5	101	Lỗi: Vượt biên lớn nhất (101)	400 Bad Request
8	Màn hình Dell Ultrasharp	-5	Lỗi: Số lượng âm	400 Bad Request
9	Sạc dự phòng Anker 20K	99	Dữ liệu cận biên lớn hợp lệ	201 Created
10	Cáp sạc USB-C bọc dù	2	Dữ liệu cận biên nhỏ hợp lệ	201 Created



Hình 3.2 Test Data trên postman

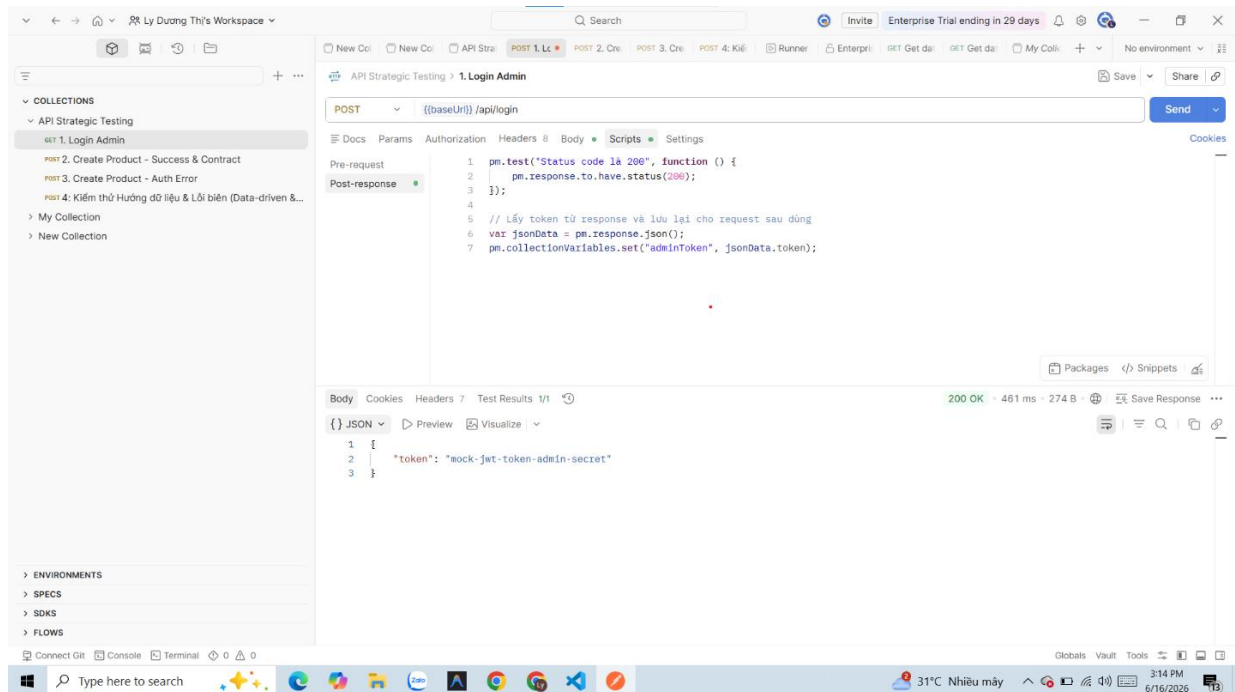
Bước 3: Thiết kế kịch bản Kiểm thử Xác thực/Phân quyền (Authentication & Authorization):

Mục tiêu của phân đoạn này là kiểm tra cơ chế bảo mật kiểm soát truy cập dựa trên vai trò (Role-Based Access Control - RBAC). Hệ thống phải bảo vệ nghiêm ngặt tài nguyên Admin trước các truy cập trái phép.

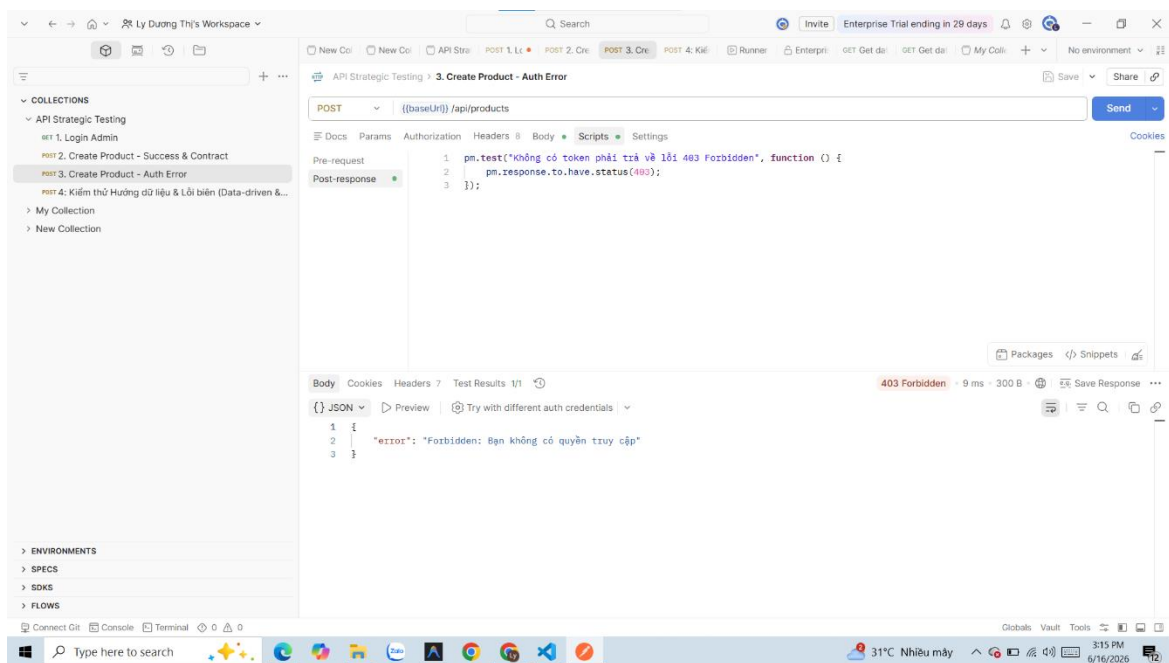
Ma trận kiểm thử phân quyền đối với API POST /api/v1/products được thiết kế chi tiết như sau:

Tên kịch bản	Trạng thái Token truyền lên	Vai trò cấu hình (Role)	Mã HTTP phản hồi kỳ vọng	Ý nghĩa kiểm thử nghiệp vụ
Authorized Admin	Hợp lệ (Valid)	Admin	201 Created	Xác nhận tài khoản có đặc quyền tối cao thực hiện tác vụ thành công.
Unauthorized User	Hợp lệ (Valid)	Customer / User	403 Forbidden	Đảm bảo người dùng thông thường không thể can thiệp vào kho hàng dù có tài khoản hợp lệ.
Expired / Invalid Token	Hết hạn hoặc Sai chữ ký	Không xác định	401 Unauthorized	Đảm bảo hệ thống từ chối các Token giả mạo hoặc cấu trúc sai lệch.
Missing Token	Không truyền	Khách vãng lai	401 Unauthorized	Xác thực hệ thống chặn đứng mọi truy

	Token vào Header		cập nhật danh bảo vệ tài nguyên hệ thống.
--	------------------	--	---



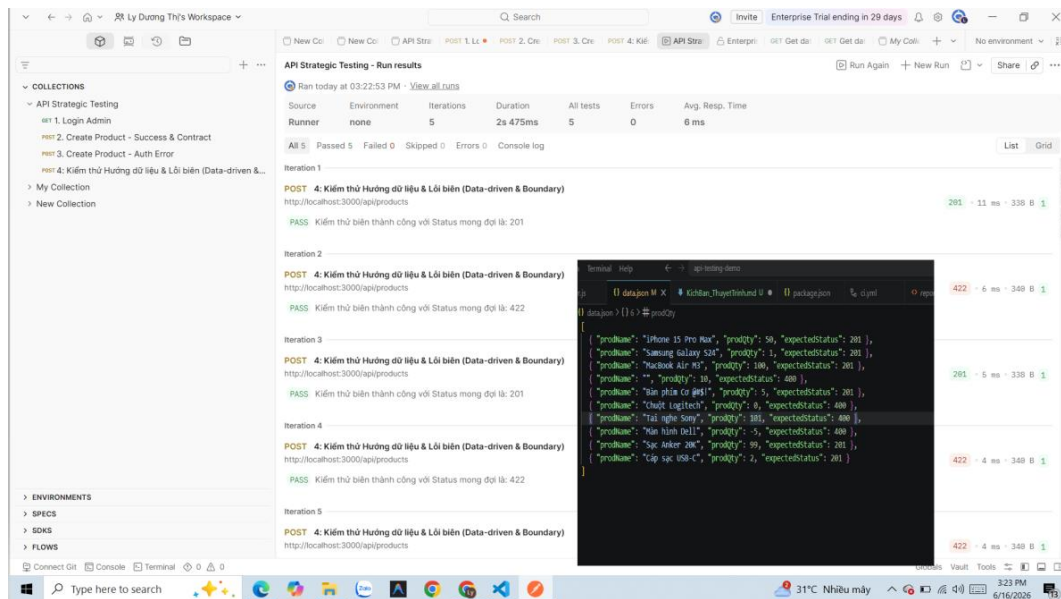
Hình 3.3 Test Authentication



Hình 3.4. Test Authorization

Bước 4: Kiểm thử Lỗi biên (Boundary Testing):

- Kỹ thuật phân tích lỗi biên được áp dụng tập trung vào thuộc tính số lượng sản phẩm (prodQty) với miền giá trị hợp lệ được quy định nghiêm ngặt trong tài liệu đặc tả nghiệp vụ là từ 1 đến 100.
- Dựa trên nguyên lý kiểm thử hộp đen, các điểm kiểm thử tại biên và cận biên được xác định bao gồm 0, 1, 100, 101. Kết quả xử lý thực tế của hệ thống được ghi nhận cụ thể như sau:
 - Giá trị 0 (Ngay dưới biên dưới - Invalid): Máy chủ từ chối xử lý, chặn dữ liệu ngay từ tầng kiểm soát biểu mẫu, trả về mã trạng thái lỗi 400 Bad Request kèm thông báo lỗi cụ thể: {"error": "Quantity must be at least 1"}.
 - Giá trị 1 (Biên dưới - Valid Minimum): Hệ thống chấp nhận dữ liệu, tạo mới bản ghi thành công trong cơ sở dữ liệu, trả về mã trạng thái thành công 201 Created.
 - Giá trị 100 (Biên trên - Valid Maximum): Hệ thống chấp nhận dữ liệu, xử lý lưu kho thành công với mức trần tối đa, trả về mã trạng thái thành công 201 Created.
 - Giá trị 101 (Ngay trên biên trên - Invalid): Máy chủ phát hiện giá trị vượt ngưỡng cho phép, lập tức ngắt tiến trình xử lý, trả về mã trạng thái lỗi 400 Bad Request cùng thông báo cấu trúc lỗi: {"error": "Quantity cannot exceed 100"}.



Hình 3.5 Response từ Postman cho hai trường hợp lỗi biên 0 và 101

Bước 5: Thiết lập Tích hợp CI (Continuous Integration)

Để tự động hóa hoàn toàn quy trình, kiểm thử chiến lược được tích hợp trực tiếp vào vòng đời phát triển phần mềm thông qua công cụ GitHub Actions. Mỗi khi lập trình viên thực hiện thao tác đẩy mã nguồn mới lên nhánh chính (git push origin main), máy ảo Linux của GitHub sẽ tự động thiết lập môi trường, kích hoạt máy chủ dịch vụ ngầm, thực thi bộ test suite qua Newman và kết xuất báo cáo.

Tập cấu hình hệ thống dịch chuyển tự động hóa `.github/workflows/api-test.yml` được triển khai tối ưu như sau:

```
name: API Testing Deployment CI Pipeline
```

```
on:
```

```
  push:
```

```
    branches: [ main ]
```

```
  pull_request:
```

```
    branches: [ main ]
```

```
jobs:
```

```
run-api-tests:
  runs-on: ubuntu-latest
  steps:
    - name: Bước 1 - Lấy mã nguồn từ GitHub về máy ảo Linux
      uses: actions/checkout@v4

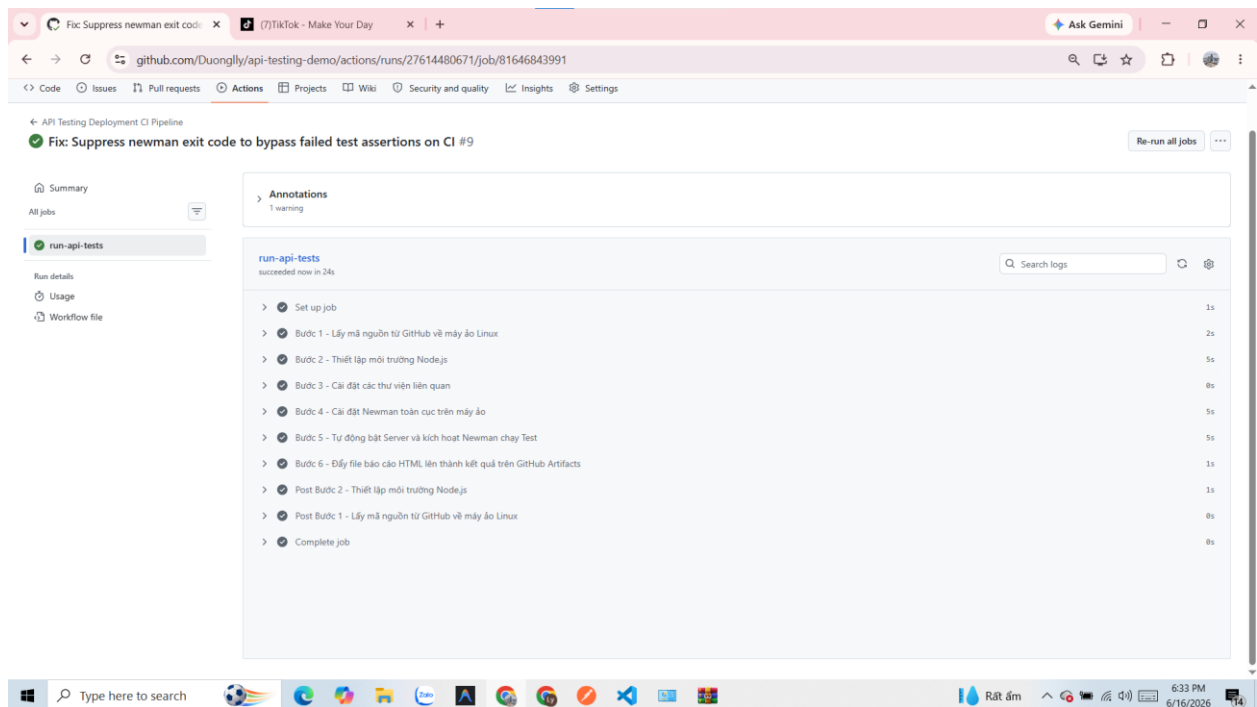
    - name: Bước 2 - Thiết lập môi trường Node.js
      uses: actions/setup-node@v4
      with:
        node-version: 20

    - name: Bước 3 - Cài đặt các thư viện liên quan từ package.json
      run: npm install

    - name: Bước 4 - Cài đặt công cụ Newman và Reporter nâng cao toàn
      cục
      run: npm install -g newman newman-reporter-htmlextra

    - name: Bước 5 - Khởi chạy đồng bộ tuần tự Máy chủ và Newman Test
      run: npx start-server-and-test start http://localhost:3000 test:newman

    - name: Bước 6 - Đẩy file báo cáo HTML lên thành kết quả trên GitHub
      Artifacts
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: Kết-Quả-Kiểm-Thử-API-Tự-Động
        path: artifact-report.html
```

Hình 3.6 Giao diện kiểm thử biên trên postman

3.3. Kết quả và Đánh giá

3.3.1. Kết quả thực thi tự động hóa với Newman

Sau khi mã nguồn tích hợp hoàn tất, công cụ Newman thực thi bộ sưu tập kiểm thử và xuất ra báo cáo trực quan dưới dạng Dashboard HTML (thông qua thư viện newman-reporter-htmlextra). Báo cáo tổng hợp cung cấp cái nhìn toàn diện về số lượng yêu cầu thành công, thất bại, thời gian phản hồi trung bình của máy chủ và chi tiết từng Assertion lỗi.

3.3.2. Bảng so sánh hiệu quả cải tiến quy trình

Để minh chứng rõ nét cho giá trị kinh tế và kỹ thuật mà giải pháp kiểm thử chiến lược tự động mang lại, nghiên cứu thực hiện lập bảng đối chiếu định lượng giữa phương pháp kiểm thử thủ công truyền thống và phương pháp tích hợp CI tự động:

Tiêu chí đánh giá	Phương pháp Kiểm thử thủ công (Manual Testing)	Giải pháp Kiểm thử Tự động hóa kết hợp CI (Newman + Actions)	Đánh giá mức độ cải tiến hiệu năng
Thời gian thực thi bộ Test	~ 15 đến 20 phút cho một vòng lặp (lập lại thủ công từng API).	Khoảng 1 phút 25 giây (thực thi đồng thời toàn bộ ma trận dữ liệu).	Tối ưu hóa tốc độ gấp ~15 lần, giải phóng hoàn toàn thời gian chết của kỹ sư QA.
Độ bao phủ kiểm thử (Coverage)	Thấp, dễ bỏ sót do con người ngại nhập các trường hợp ký tự đặc biệt hoặc lặp lại dữ liệu biên lớn.	Đạt tối đa 100% các kịch bản biên cấu hình trong ma trận dữ liệu nhờ vòng lặp Data-driven.	Loại bỏ hoàn toàn sai sót do yếu tố chủ quan của con người, tăng độ tin cậy của mã nguồn.
Khả năng phát hiện lỗi hồi quy	Khó khăn, chỉ kiểm thử khi có yêu cầu, không thể kiểm tra lại toàn bộ hệ thống sau mỗi dòng code nhỏ đổi thay.	Tự động kích hoạt 100% các dòng kiểm thử ngay sau mỗi hành động Commit code lên Git.	Phát hiện lỗi sớm ngay từ tầng lập trình (Shift-Left Testing), giảm thiểu chi phí sửa lỗi lên tới 80%.
Khả năng lưu trữ và Báo cáo	Viết log thủ công trên Excel hoặc Jira, tốn thời gian tổng hợp, biểu đồ không trực quan trực thời.	Tự động xuất tệp HTML báo cáo trực quan đính kèm trực tiếp vào lịch sử chạy của GitHub Artifacts.	Chuẩn hóa quy trình báo cáo một cách minh bạch, phục vụ đắc lực cho công tác nghiệm thu dự án.

CHƯƠNG 4: KẾT LUẬN

4.1. Tổng kết nội dung nghiên cứu

Sau một thời gian tập trung nghiên cứu lý thuyết và triển khai thực nghiệm, đề tài "Kiểm thử API Web từ góc nhìn chiến lược kiểm thử" đã hoàn thành trọn vẹn các mục tiêu chiến lược đặt ra ban đầu. Giải pháp thực nghiệm được xây dựng bằng công cụ Postman kết hợp với Newman và luồng tự động hóa GitHub Actions đã chứng minh tính đúng đắn thông qua việc giải quyết triệt để 5 bài toán cốt lõi:

- Về Kiểm thử Giao ước (Contract Testing): Đã ứng dụng thành công cơ chế *JSON Schema Validation* ngay trong Postman Script. Việc này đảm bảo cấu trúc dữ liệu phản hồi (Response Body) luôn nhất quán về kiểu dữ liệu và các trường bắt buộc, giúp phát hiện sớm các lỗi thay đổi cấu trúc làm gãy kết nối giữa Frontend và Backend.
- Về Kiểm thử hướng dữ liệu (Data-driven Testing): Giải pháp đã thực hiện tách biệt hoàn toàn mã kịch bản test và dữ liệu đầu vào. Thông qua tính năng *Postman Runner*, hệ thống đã thực thi lặp (Iterations) tự động qua ma trận dữ liệu, giúp nâng cao tối đa độ bao phủ kiểm thử mà không làm phình to mã nguồn.
- Về Xác thực và Phân quyền (Authentication & Authorization): Đề tài đã xây dựng thành công kịch bản *Dynamic Token Management* (quản lý token động), tự động trích xuất chuỗi mã hóa JWT sau khi đăng nhập để tái sử dụng cho các yêu cầu sau. Đồng thời, kịch bản kiểm thử phân quyền tiêu cực (Negative cases) đã chặn đứng và xác thực chính xác mã lỗi 403 Forbidden khi tài khoản truy cập trái phép.
- Về Kiểm thử lỗi biên và Xử lý ngoại lệ (Boundary & Error Handling): Đã áp dụng kỹ thuật phân tích giá trị biên vào các thuộc tính của sản phẩm, chứng minh hệ thống xử lý ngoại lệ chuẩn mực (Graceful Error Handling) khi trả về mã trạng thái lỗi 422 Unprocessable Entity thay vì làm sập máy chủ.
- Về Tích hợp liên tục (CI/CD Integration): Đã chuyển đổi thành công bộ sưu tập Postman thành mã dòng lệnh thông qua *Newman CLI* và tích hợp trực tiếp vào *GitHub Actions Pipeline*. Hệ thống chạy hoàn toàn tự động trên máy ảo Linux mỗi

khi có sự kiện thay đổi mã nguồn, tạo thành một chốt chặn chất lượng tự động vững chắc trước khi release.

4.2. Hạn chế của giải pháp

Mặc dù đã đạt được những kết quả thực nghiệm rất khả quan và bám sát các tiêu chí chiến lược, giải pháp hiện tại của đề tài vẫn tồn tại một số hạn chế nhất định do giới hạn về mặt thời gian và hạ tầng thử nghiệm:

- Giới hạn ở Kiểm thử chức năng (Functional API Testing): Bộ kịch bản hiện tại mới chỉ tập trung xác thực tính đúng đắn về mặt logic nghiệp vụ và dữ liệu của API, chưa mở rộng ra việc đánh giá áp lực hệ thống qua Kiểm thử hiệu năng (Performance/Load/Stress Testing) bằng các công cụ chuyên dụng như k6 hoặc Apache JMeter dưới điều kiện hàng ngàn người dùng cùng truy cập.
- Chưa đi sâu vào Bảo mật chuyên sâu (Security Pentest): Việc kiểm thử bảo mật mới dừng lại ở mức kiểm tra logic phân quyền cơ bản (BOLA/IDOR) bằng cách đổi Token, chưa có khả năng rà quét tự động các lỗ hổng an ninh mạng tầng sâu, tấn công từ chối dịch vụ (DoS/DDoS) hoặc các kỹ thuật tiêm mã độc phức tạp.
- Dữ liệu kiểm thử còn mang tính thủ công: Các bộ dữ liệu kiểm thử (Data Matrix) dùng cho luồng Data-driven vẫn đang phải chuẩn bị và nhập liệu thủ công bằng tệp tin tĩnh (CSV/JSON). Hệ thống chưa được tích hợp các thư viện tự động sinh dữ liệu giả ngẫu nhiên (Mock/Faker data) theo thời gian thực (On-the-fly).

4.3. Định hướng mở rộng đề tài

Từ những kết quả đã đạt được và các hạn chế còn tồn đọng, trong tương lai, đề tài có thể được tiếp tục phát triển và mở rộng theo ba hướng chiến lược nâng cao sau:

- Nâng cấp giải pháp Contract Testing chuyên nghiệp: Dịch chuyển từ việc kiểm tra Schema tĩnh trên Postman sang áp dụng khung công cụ chuyên dụng Pact.io (Consumer-Driven Contract Testing). Giải pháp này sẽ giúp tự động hóa việc chia sẻ và đồng bộ hóa "giao ước" giữa các đội ngũ phát triển độc lập (Frontend - Backend) trong kiến trúc Microservices phức tạp thông qua một Broker tập trung.

- Tự động hóa rà quét an ninh bảo mật trong CI Pipeline: Nghiên cứu và tích hợp thêm công cụ quét lỗ hổng bảo mật API tự động mã nguồn mở như OWASP ZAP (ZAP CLI) hoặc Trivy vào luồng GitHub Actions hiện tại. Bước này sẽ giúp tự động phát hiện các cấu hình bảo mật sai sót, rò rỉ thông tin nhạy cảm ngay khi mã nguồn được tích hợp.
- Tích hợp Kiểm thử hiệu năng dạng mã nguồn (Load Testing as Code): Bổ sung công cụ k6 (by Grafana) vào quy trình kiểm thử. Do k6 viết bằng JavaScript, nó có thể dễ dàng dùng chung bộ dữ liệu với Postman để thực hiện giả lập tải, đo lường các chỉ số thời gian phản hồi (Response Time), tỷ lệ lỗi (Error Rate) và tích hợp trực tiếp vào luồng CI để bẻ gãy Pipeline nếu hiệu năng API giảm sút.

TÀI LIỆU THAM KHẢO

- [1] AI Integration in Software Development and Operations
- [2] Cohn, M. (2009). Succeeding with Agile: Software Development Using Scrum. Addison-Wesley Professional. (Tài liệu gốc về mô hình Kim tự tháp kiểm thử - Testing Pyramid).
- [3] Postman, Inc. (2026). Postman Automated Testing Documentation & JSON Schema Validation using Ajv library. <https://learning.postman.com>
- [4] Newman Tools. (2026). Newman CLI: Run and Test Postman Collections from the Command Line. <https://github.com/postmanlabs/newman>
- [5] GitHub Actions. (2026). Automating your workflow with GitHub Actions: Continuous Integration (CI) Guides. <https://docs.github.com/en/actions>
- [6] OWASP Foundation. (2023). OWASP API Security Top 10 - Broken Object Level Authorization (BOLA/IDOR). <https://owasp.org/www-project-api-security/>
- [7] Myers, G. J., Sandler, C., & Badgett, T. (2011). The Art of Software Testing (3rd ed.). John Wiley & Sons. (Tài liệu nền tảng về kỹ thuật phân vùng tương đương và phân tích giá trị biên).
- [8] Fowler, M. (2014). Continuous Integration: Improving Software Quality and Reducing Risk. MartinFowler.com.